

Roofline-Aware LLM Inference FlashAttention + 4-bit Quantization

Llama 3.1 8B Instruct on NVIDIA L4: FlashAttention improves speed, while 4-bit quantization cuts memory.

Team: Nathan Gong · ng2695 | Jiaxiong Hu · jh4878 | Fu Liu · fl2597 |
Jason Zeng · jlz2128

GitHub: https://github.com/nathan-gong/hpml_project
Tracking: wandb.ai/hpml2026-final-project/hpml2026-final-project

Motivation & Problem

Why optimize inference? Separate prefill vs. decode bottlenecks.

Why this matters

- Single-GPU LLM serving is constrained by first-token latency, decode speed, and memory capacity.
- Llama 3.1 8B nearly fills an NVIDIA L4 in FP16, so memory optimization is practically important.
- Prefill and decode stress hardware differently, so one aggregate latency number hides the bottleneck.

Problem statement

- Study four inference paths: FP16 baseline, FlashAttention, 4-bit quantization, and FlashAttention + quantization.
- Evaluate sequence lengths 128, 256, 512, 1024, 2048; treat 128 as warmup / short-context point.
- Use roofline analysis to explain whether speed and memory changes come from arithmetic intensity or kernel efficiency.

Experimental Setup — Model, Inputs, Hardware

Controlled single-GPU inference study.

Model

- Llama 3.1 8B Instruct
- HF checkpoint: meta-llama/Meta-Llama-3.1-8B-Instruct
- Decoder-only autoregressive transformer

Inputs

- Deterministic prompt benchmark
- Exact token lengths: 128–2048
- Inference only; no training labels

Hardware & tools

- 1× NVIDIA L4 (24 GB) on GCP
- PyTorch + Transformers + bitsandbytes
- W&B + torch.profiler; roofline: 121 TFLOP/s, 300 GB/s

Technical Approach — Four Configurations

Only precision mode and attention backend change; benchmark path is shared.

1. **Baseline: FP16 + standard SDPA/math**

Reference path for latency, throughput, peak memory, KV-cache, and roofline placement.

2. **FlashAttention: FP16 + FlashAttention backend**

Targets attention-kernel efficiency; expected to help long-context prefill and achieved FLOP/s.

3. **Quantized: 4-bit + standard SDPA/math**

Weight-only quantization reduces model memory and increases arithmetic intensity.

4. **FA + Quantized: 4-bit + FlashAttention backend**

Tests whether efficient attention and smaller weights are complementary, and whether dequantization overhead limits gains.

Implementation & Profiling Methodology

Modular code path + phase-isolated measurements.

Implementation modules

- quantization.py: FP16 / 4-bit model loading through one interface
- metrics.py: CUDA memory snapshots and peak tracking
- kv_cache_utils.py: computes KV-cache bytes from past_key_values
- profiling.py + roofline.py: profiler-driven FLOPs/bytes/roofline metrics

Metrics captured

- Prefill latency (TTFT) and decode tokens/sec
- Prefill/decode peak GPU memory and KV-cache MB
- Estimated FLOPs, arithmetic intensity, achieved FLOP/s
- W&B system telemetry: GPU memory/utilization/power, CPU, disk, network

Results — Headline Numbers

Profiler-based seq=2048 results; 128-token case treated as warmup / short-context.

1.60x

faster inference

prefill faster

1668.21 → 1043.31 ms

(seq=2048, baseline → FlashAttention)

61.6%

less GPU memory

less peak decode memory

16.19 → 6.22 GB

(baseline → FA + Quant, seq=2048)

2.4x

higher arithmetic intensity

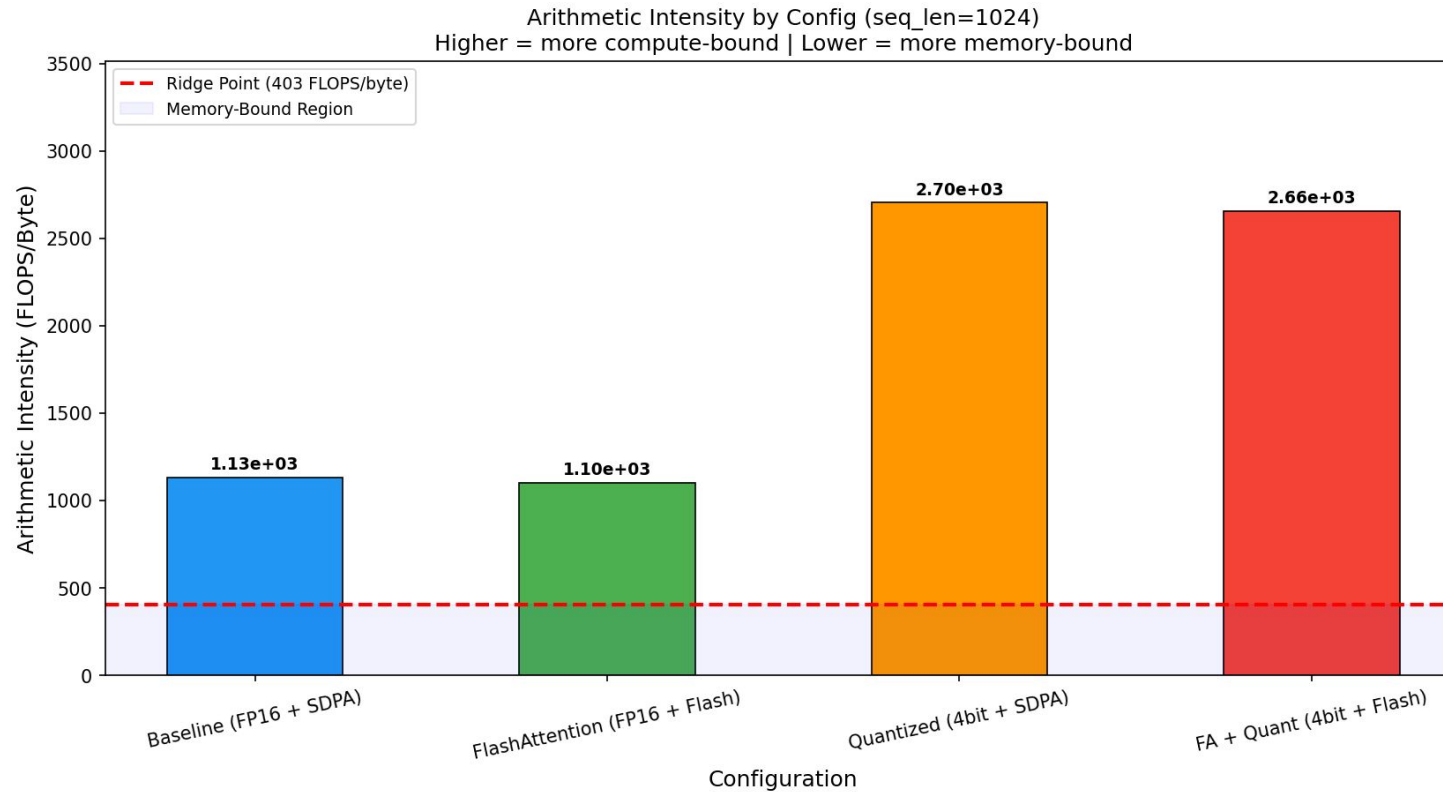
2065 → 4962 FLOPs/byte

(baseline → FA + Quant, seq=2048)

Main takeaway: FlashAttention is the speed optimization; 4-bit quantization is the memory-capacity optimization.

Results — Arithmetic Intensity

Quantization shifts the workload rightward in the roofline model.

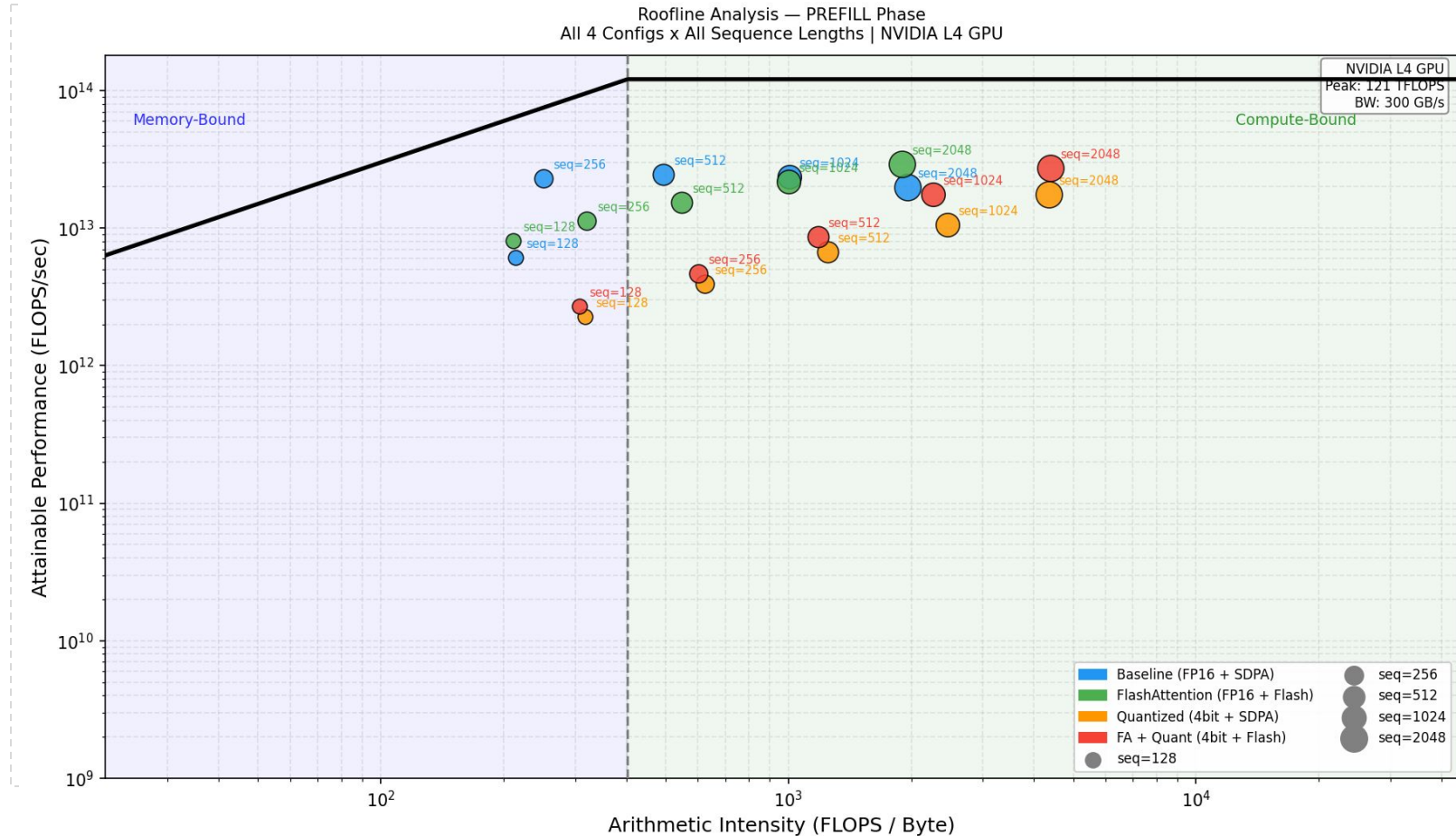


Takeaway

- At seq=1024, quantized configs are ~2.4× higher in FLOPs/byte than FP16 configs.
- FlashAttention is close to baseline on the x-axis: its benefit is kernel efficiency, not weight-memory reduction.
- Higher arithmetic intensity alone does not guarantee faster runtime.

Profiler Evidence — Prefill Roofline

Prompt processing is attention-heavy; long contexts expose FlashAttention benefits.

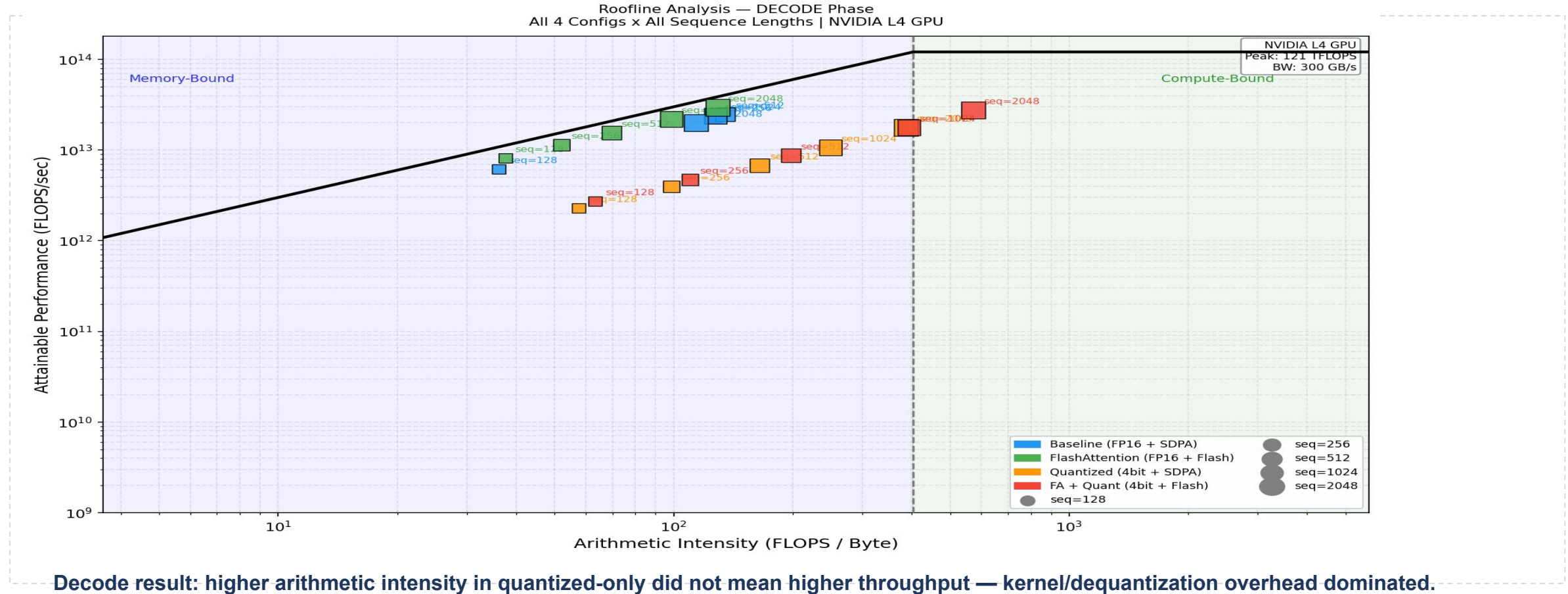


What to point at

- Longer prompts move points up/right: more compute and better GPU utilization.
- FlashAttention raises achieved performance at long context.
- Quantized-only moves right but can remain slower due to dequantization / kernel overhead.

Profiler Evidence — Decode Roofline

Autoregressive decode is more memory-sensitive than prefill.



Interpretation — What the Results Mean

Keep the explanation phase-aware: prefill $\neq$ decode.

What worked

- FlashAttention improved achieved performance and long-context speed.
- 4-bit quantization cut peak GPU memory by more than 60%.
- KV-cache tracking showed weight-only quantization does not reduce activation cache memory.

What surprised us

- Quantized-only decode was slower than FP16 despite higher arithmetic intensity.
- FA + Quantized was the best memory/performance tradeoff, not always the fastest path.
- The 128-token point behaved like a warmup / fixed-overhead regime; 256–2048 drives conclusions.

Limitations & Next Steps

What we would improve with more time.

- Limitations: one GPU type (L4), one model, batch size 1, estimated roofline rather than Nsight hardware-counter roofline.
- Quantization is weight-only; KV-cache quantization was not implemented.
- Next: test KV-cache quantization, vLLM/PagedAttention, TensorRT-LLM, batch-size scaling, and A100/H100.
- Add explicit warmup/repeats for all runs and validate FLOPs/bytes with Nsight Compute.

Teamwork & Contributions

Required contribution slide — who owned what.

Member	Primary contribution
Nathan Gong	Repo integration, FlashAttention execution, FlashAttention control, latency metrics, Latex editing
Jiaxiong Hu	Project idea proposition, quantization integration, memory profiling, KV-cache tracking, validation sweeps, draft presentation and paper
Fu Liu	System profiling, experiment orchestration, roofline analysis, W&B dashboards and visualization, result consolidation
Jason Zeng	GCP environment setup, model/data pipeline, baseline definition, prefill and decode verification

Collaboration: shared GitHub repo + W&B workspace; weekly syncs; final claims verified against JSON results and W&B figures.

Thank you
Questions?